

Hydrophone Graphical Interface User Guide

Summary:

1. How to launch the interface	3
2. Explanation of interface	3
3. Understanding scan settings	4
1. Point mode	5
2. Length mode	5
3. Matrix mode	6
4. Scanning interface	6
5. Environment setup (installation)	8
6. To go further	10
1. Common bugs	10
2. Editing other settings	14

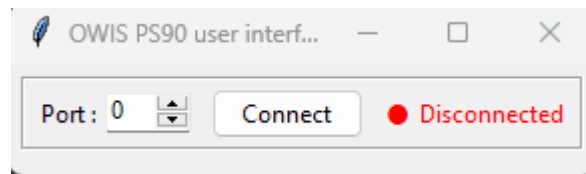
Folder structure

```
.
├── hydrophone_handling/
├── graphic_interface.py
├── Hydrophone Graphical Interface User Guide.docx
├── classes/
│   ├── HDO4034A/
│   │   └── HDO4034A.py
│   ├── ps90/
│   │   ├── ps90.dll
│   │   ├── OWIS_PS90.py
│   │   ├── LinearStageL84N.py
│   │   └── RotaryStageDMT65.py
│   ├── display_time.py
│   └── path_import.py
├── images/
│   ├── img_scan_length.png
│   └── img_scan_point.png
├── matrix_csv/
│   └── example.csv
├── out/
├── scan/
│   ├── scan_by_interf.py
│   ├── scan_for_matrix.py
│   ├── scan_classic.py
│   └── utils/
│       ├── axe_def.py
│       ├── break_timer.py
│       ├── osc_scan_settings.py
│       ├── scan_viewer.py
│       └── write_csv.py
├── Drivers/
│   ├── CDM2123620_Setup.exe
│   ├── mauiiinstaller64_11.5.0.7.exe
│   ├── activedso-developer-guide_03aug22.pdf
│   └── activedsoinstaller_2.43.exe
└── README.md
```

1. How to launch the interface

The interface is simply a Python script, so you only need to run the following file: `C:/.../hydrophone_handling/graphic_interface.py`. Besides, you must be connected to PS90 engine (via USB) and if you want to perform a scan, you must also be connected to the oscilloscope (via ethernet).

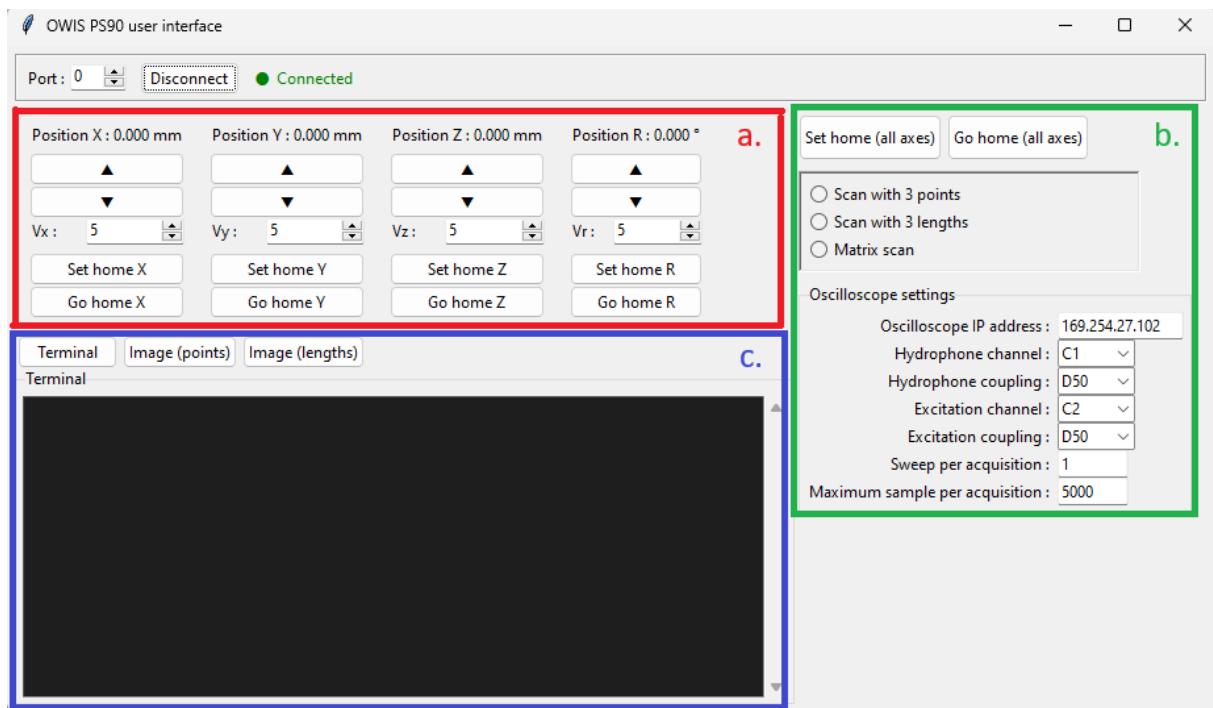
If the interface has been launched correctly, you should see something like this (it may take few seconds).



Next, to open the full interface, you have to

- Connect to PS90(+) engine (via USB).
- Select a port (0 = default: automatically detects the correct USB port).
- Click the "Connect" button once.

2. Explanation of interface panels



a. Axes control:

You can control all the axes from this panel.

Position {axis}: allows you to set the position of the axis and displays its current position.

V{axis}: allows you to set the manual movement speed of the axis.

Set / Go home buttons: perform these actions to the selected axis only.

b. Scan settings:

Here, the Set and Go home buttons apply to all axes.

(Note: the remaining settings are only used for scanning.)

You can select the scan mode.

(All scan modes are explained in the next section.)

c. Terminal / images:

You can choose what to display:

- Terminal output (stdout + stderr)
- Images from the scanning environment.

The terminal displays:

- Errors: “communication error”
- Successfully executed functions
- Some scan information
- etc

Warning: if the PS90 engine is disconnected for any reason other than clicking the “Disconnect” button, the connection status displayed in the interface will not be updated.

3. Understanding scan settings

From this interface, you can launch a scan in three different ways.

If you want to scan:

- a **rectangular area**, use **point** or **length mode**.
- a more **specific area**, use the **matrix mode**.

Before launching your scan, you have to select your oscilloscope settings.

Be aware of the “Maximum sample per acquisition” setting: if it is set too high, the scan will require a large amount of storage.

3.1. Point mode

You can define three points (P1, P2 and P3) as shown in the figure: they define the rectangular area that the engine will scan.

Clicking “Set P1” records the current position as P1.

dX / dY / dZ: This is the distance (in millimetre) between two points in the specified axis.

!!\ WARNING: you must perform **set home before defining the points**. Otherwise, the scan will not cover the intended area. Once the points have been defined, do not perform a set home again.

3.2. Length mode

You can specify three lengths (in millimetre) (X', Y' and Z') as shown in the figure. To define the rectangular area that the engine will scan.

Warning:

$$X' = - \text{dist} * \vec{x} ; Y' = - \text{dist} * \vec{y} ; Z' = + \text{dist} * \vec{z}$$

dX / dY / dZ: This is the distance (in millimetre) between two points in the specified axis.

!!\ WARNING: Perform set home at the position marked by the red cross before starting the scan (otherwise, the scan will start from the current home position).

3.3. Matrix mode

For this mode, you need to create a CSV file containing the coordinates that the engine should move to.

Using the following format:

[Line 1]: X_1, Y_1, Z_1, R_1

[Line 2]: X_2, Y_2, Z_2, R_2

[Line 3]: X_3, Y_3, Z_3, R_3

...

(R coordinate is not mandatory)

And it will capture a point in (X_1, Y_1, Z_1, R_1) then in (X_2, Y_2, Z_2, R_2) , then in (X_3, Y_3, Z_3, R_3) , then in ...

(Note: A tutorial explaining how to create a CSV file for this mode is available [here](#))

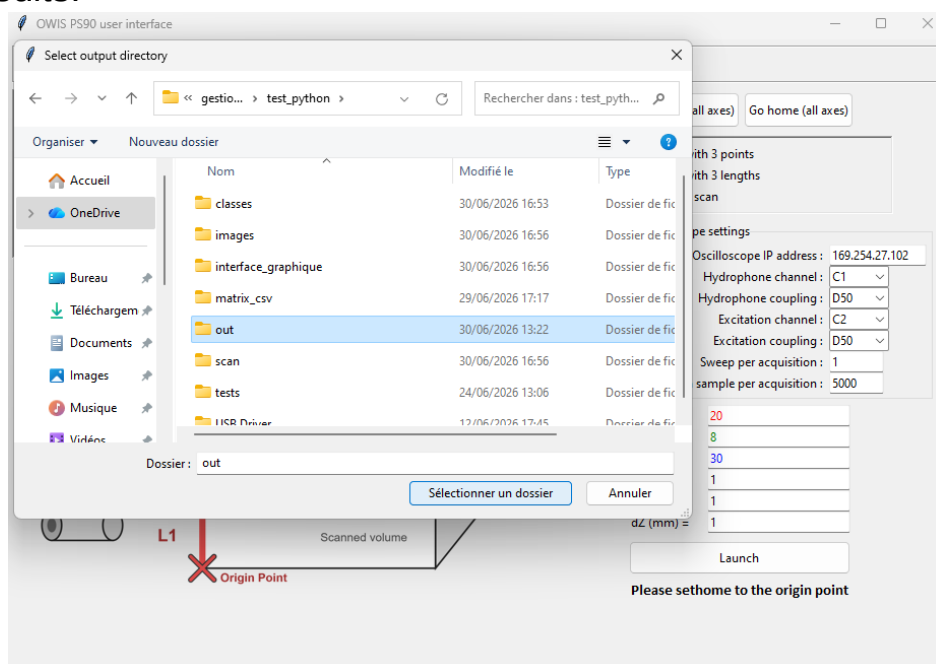
Example CSV file:

`./hydrophone_handling/matrix_csv/example.csv`

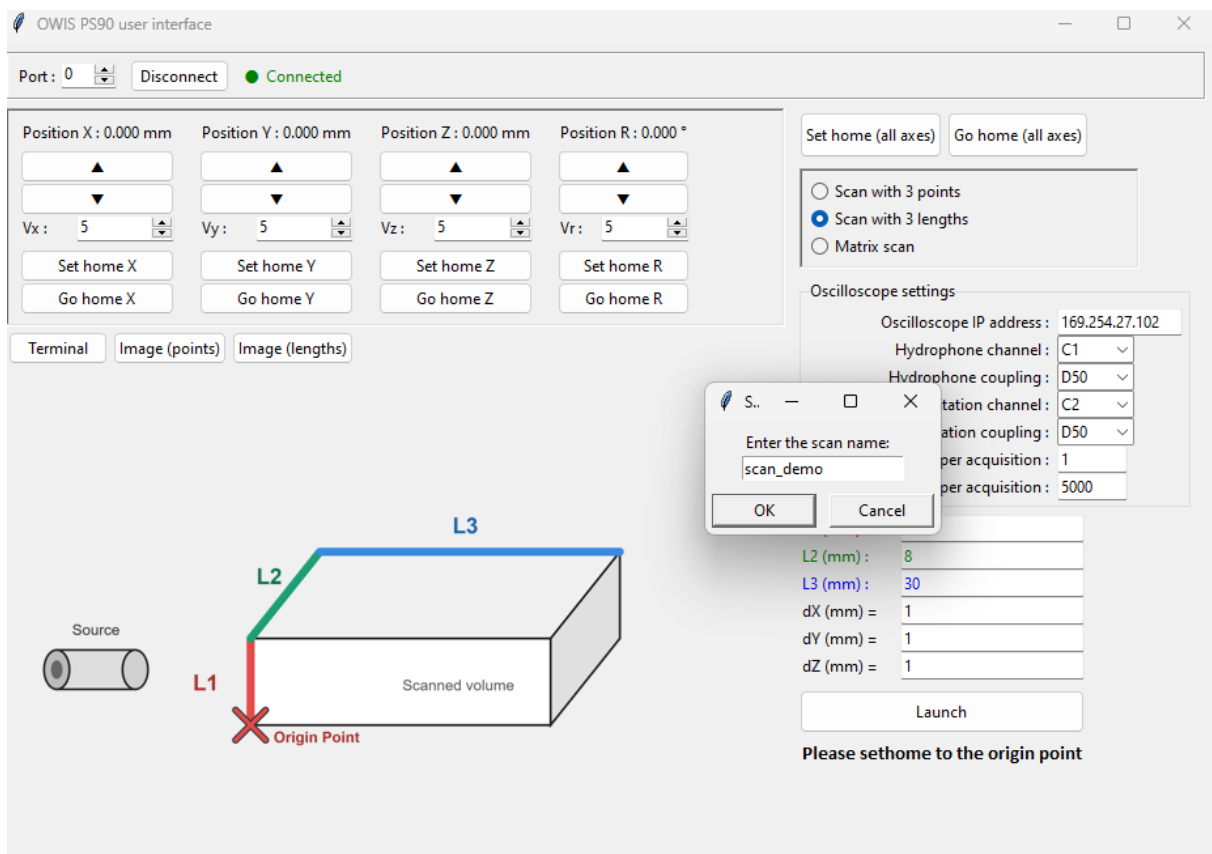
The screenshot shows a settings window for 'Matrix scan' mode. It includes three radio buttons: 'Scan with 3 points', 'Scan with 3 lengths', and 'Matrix scan' (which is selected). Below these are 'Oscilloscope settings' with fields for 'Oscilloscope IP address' (169.254.27.102), 'Hydrophone channel' (C1), 'Hydrophone coupling' (D50), 'Excitation channel' (C2), 'Excitation coupling' (D50), 'Sweep per acquisition' (1), and 'Maximum sample per acquisition' (5000). At the bottom, there is an 'Import a CSV file' button, a status 'No file selected', and a 'Launch' button.

4. Scanning interface

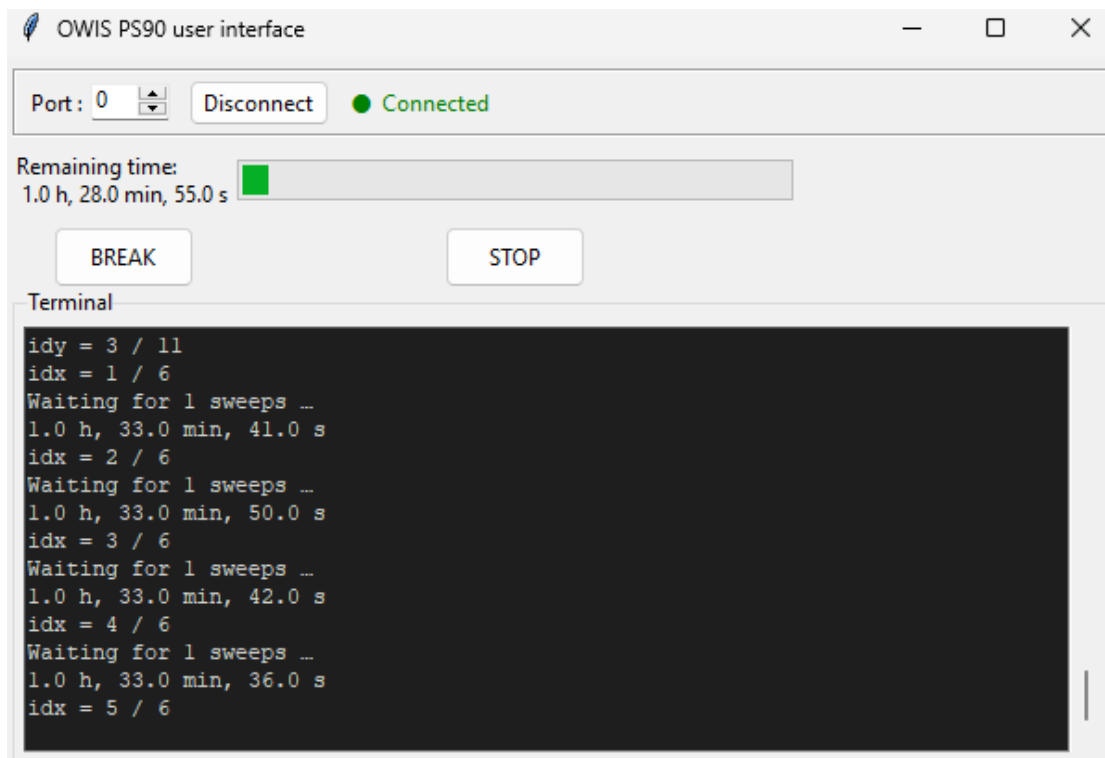
Once you have entered all the desired settings, click the “Launch” button. A new window will open, prompting you to select an output folder for the scan results.



Once you have selected your output folder, enter a name for the scan:

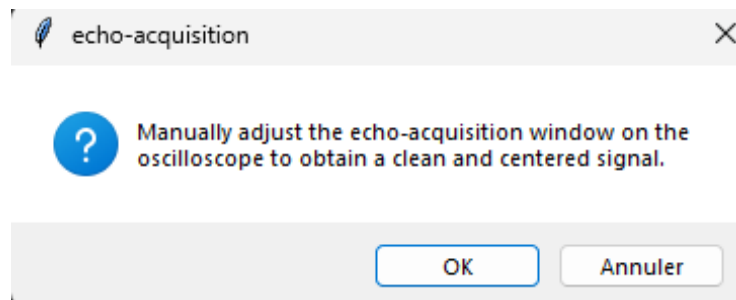


And after that, the scan will start:



To monitor the progress of the scan, watch the terminal:
It displays information such as: connection status, the estimated total scan duration (the estimate is inaccurate at the beginning but becomes more reliable as more points are acquired), and other status messages.

Before leaving scan to run for several hours, configure the required settings on the oscilloscope (explained in the window).



You can also pause or a stop scan if you want.

To access the scan results, you have to find your output folder and, in this folder, a new folder will be created named after your scan.
For each measurement point, the scan creates a CSV file using the following format:

First row: X, Y, Z and R coordinates

Other rows: signal value, time

```
1  0.0,0.0,0.0,0.0
2  0.00023753,-5.0019e-06
3  0.00024302,-4.9819e-06
4  0.00024783,-4.9619e-06
5  0.00025195,-4.9419e-06
6  0.00025263,-4.9219e-06
7  0.00025126,-4.9019e-06
8  0.00024783,-4.8819e-06
```

The results can then be visualized using a Python script.
(Note: A tutorial explaining how to create a Python script to visualize the results: available [here](#))

5. Environment setup (installation)

To use this interface, you need the entire hydrophone_handling folder, including all of its files. Since the project contains several Python files, you must have **Python** installed (make sure Python is added to your

system PATH. If it is not, watch a YouTube video to add it). You must also have **pip** installed.

(Note: if you don't know how to install them, there are good tutorials on YouTube :D)

If you have just installed pip, close and reopen your terminal before continuing.

Next, install the required Python libraries:

- Launch terminal
- Copy the next command
- Press enter

Command to copy:

```
pip install pillow pywin32 numpy matplotlib
```

If an error occurs: copy it on internet. The most common cause is that pip is not installed or is not available in your PATH.

One final component still needs to be installed. If you are on windows, open the Drivers folder and run:

- activedsoinstaller_2.43.exe *(if not already installed)*
- mauiinstaller64_11.5.7.exe *(if not already installed)*
- CDM2123620_Setup.exe *(if you use a PS90+)*

If you use a PS90+, after having installed CDM2123620_Setup, open the device manager while the PS90+ controller is connected via USB. And follow the next instructions:

- Select the PS90+ device in the app,
- Click on Update button,
- Click on "Search on my computer"
- Click on "Choose from a list"
- And select USB Serial port (COM..) or FTDI

(Note: if you are not on windows, you can go to the following [link](#) and install the correct version of the driver.)

For the final step, you need the absolute path to the hydrophone_handling folder (if you don't know how to get it, search on internet).

Open `hydrophone_handling/classes/path_import.py`.

Replace the path constant with the location of hydrophone_handling folder on your computer.

Warning: To write a path, use forward slashes "/" instead of backslashes "\".

Correct: `C:/users/user_example/Documents/hydrophone_handling`

Incorrect: `C:\users\user_example\Documents\hydrophone_handling`

You are now ready to use the interface!

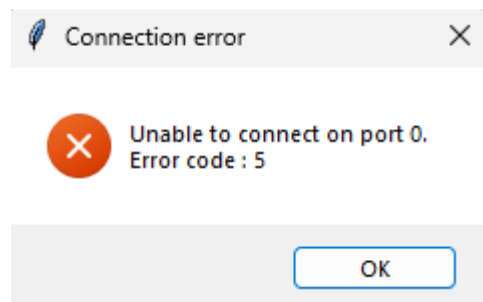
6. To go further

6.1. Common bugs

Explained bugs in this doc:

- Driver not installed (check if it is installed)
- Cable reset
- Communication error while launching a scan
- Scan crashes in the middle
- Interface crashes after a disconnection
- Break / Stop -> an error is printed
- Error print while moving axis manually

Cable reset



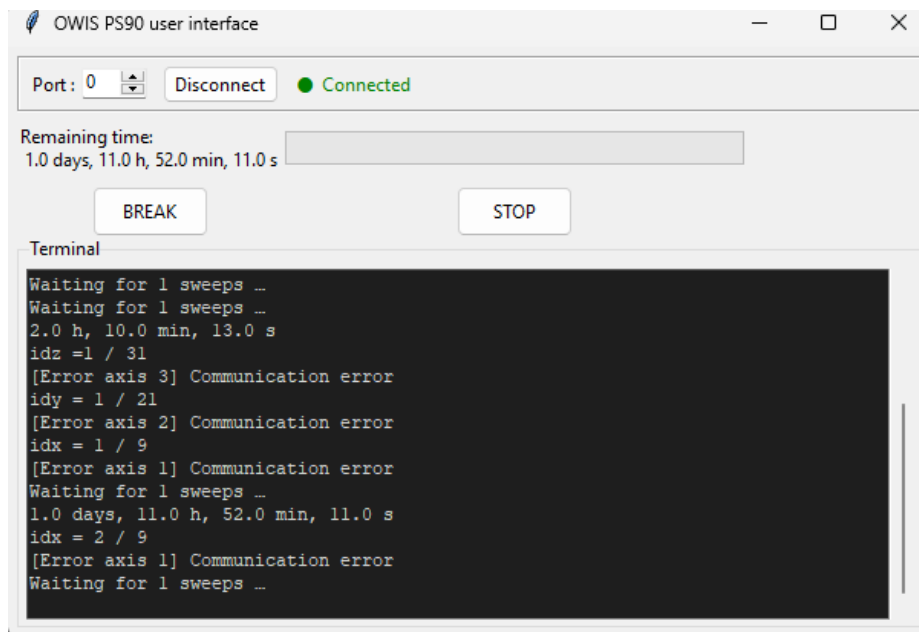
Sometimes, when you click the Connect button, it returns “Error code: 5”, in theory, this means the USB cable is not properly connected.

But in reality, it is just a small bug. You can simply click OK and then click the Connect button again.

If the issue persists, try unplugging and reconnecting the USB cable, or reboot the controller. Or if it still does not work, you can check whether the USB driver is installed (see [here](#)).

(Note: even if the USB driver is installed, there could have been an error during the installation because of antivirus software, so you can try reinstalling the driver.)

Communication error / Driver not installed

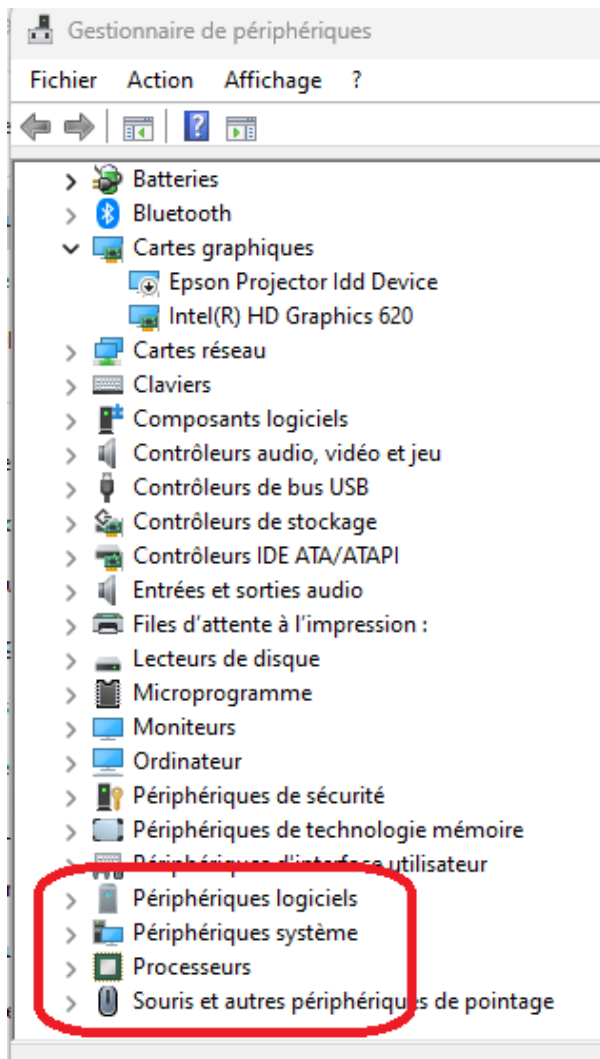


Sometimes, when you want to interact with the controller (to move axes manually or during a scan for example), terminal prints: “[Error axis n] Communication error”.

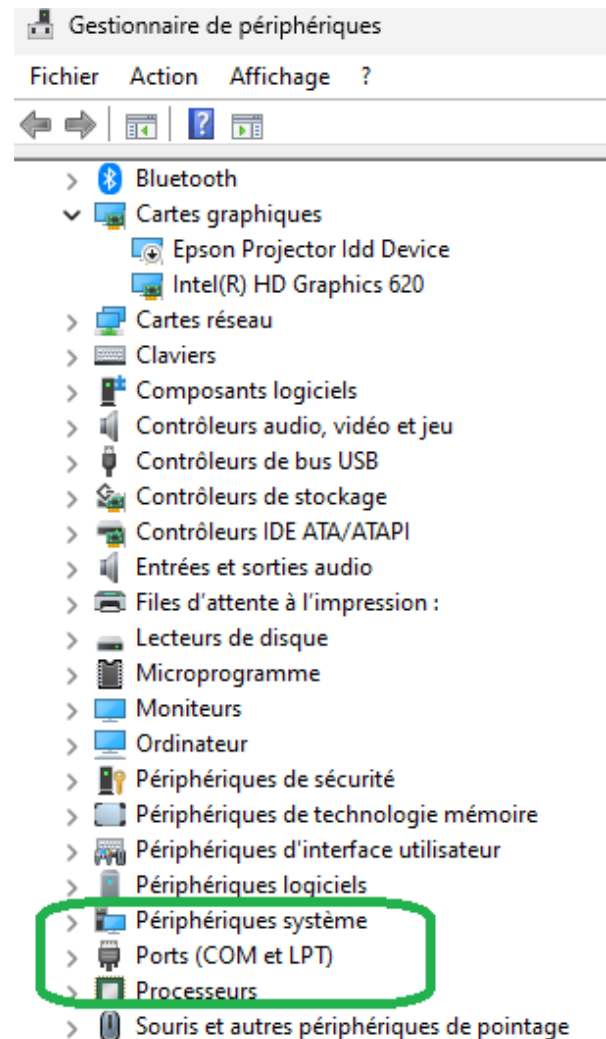
If it happens:

- during a scan: you should probably reset your scan (so click Stop and relaunch the interface).
- While trying to move axes manually: click the Disconnect button then on Connect button.

However, if the issue persists, it is possible that the USB driver (called FTDI: [website](#)) is not installed.



Driver not installed



Driver installed

To check whether it is installed, launch the “Device manager” while the PS90 controller is connected via USB, and check whether “Ports (COM et LPT)” exists. To install the driver, execute: hydrophone_handling\USB Driver/CDM2123620_Setup.

(Note: administrator permissions are required.)

Scan crash during the process

For several reasons, your scan may crash during the process.

In this case, you should not have to redo everything: the scan saves the data continuously, so you can see which coordinates have already been travelled and restart the scan from wherever you want (preferably with

different settings and a **different scan name** to prevent overwriting the existing files).

Interface crashes after trying to disconnect / reconnect

```
[PS90] Connection successfully established to the PS90 control unit
[PS90] Redefining the origin positions for all axes
object address : 000001B81B8DBAF0
object refcount : 3
object type : 000001B8108FCDF0
object type name: TclError
object repr : TclError('invalid command name ".!app.!frame.!terminalframe.!text"')
lost sys.stderr
```

You may have tried to disconnect and reconnect the PS90 controller, and the interface crashed with this message.

I do not know the exact reason, but if you relaunch the interface, it works again.

Error printed after a break / stop

If an error is printed after pressing Break or Stop, but everything still works normally, don't worry.

It is probably just the interface trying to update its data (it does this every 200 ms).

Error print while moving axis manually

```
Error occurred while refreshing control ..
```

If this message appears, it is just because the interface tries to update the position every 200 ms, and a temporary disconnection may have occurred at the same time.

In this case, this message is displayed, and the interface will wait until the next refresh (200 ms later) to update the position.

6.2. Editing other settings

The following edits are explained in this doc (stars = estimated difficulty):

- Edit oscilloscope settings (★☆☆☆☆),
- Edit scan information (★☆☆☆☆),
- Changing default values in the interface (★☆☆☆☆),
- Add/del new oscilloscope setting (★★★★☆),
- Create a script to write a CSV file (for matrix mode) (★★☆☆☆),
- Visualize a scan result (★★☆☆☆),
- Editing active axes and their IDs (★★☆☆☆),
- Launch a scan without the interface (★★★★☆),

Edit oscilloscope settings ★☆☆☆☆

To edit oscilloscope settings during a scan, you can go to `hydrophone_handling/scan/utils/osc_settings.py`.

The function is called at the beginning of the scan (just before the first movement), and all oscilloscope settings are configured here.

To adjust some settings, this file uses a specific class which is defined here: `hydrophone_handling/classes/HDO4034A/HDO4034A.py`.

For the syntax, you can inspire you from the rest of the code but globally:

- Set a new setting to a specific value:
`osc.new_setting = value`
- Get a value of a specific setting:
`value = osc.specific_setting`

(PS: to see all available settings and their type, you must go to `HDO4034A.py`)

Edit scan information ★☆☆☆☆

To edit scan information, you can go to:

- for **point** or **length mode**:
`hydrophone_handling/scan/scan_by_interf.py`
- for **matrix mode**:
`hydrophone_handling/scan/scan_for_matrix.py`

- others (without interface):
hydrophone_handling/**scan scan_classic.py**

Once you are in the correct file, you can add **print()** statements.
(Example: after connection to the oscilloscope: `print("osc connected")`)
PS: if you want to print the value of a variable in python:
`print(f"variable")`.

It is possible that you want to make a print when the code executes a specific function (i.e. `osc_setting()`), in this case you can search the function in `scan/utils`.

Changing default value in the interface ★☆☆☆☆

Open `hydrophone_handling/graphic_interface.py`.

Go to the correct frame:

- for oscilloscope settings: search `OscSettings` class
- for other scan settings: search `ScanPointFrame` class
- if it is a more specific setting (for a particular mode: search the class of the mode)

Once you have found the correct frame, look for the variable in the class's **_build function**. Normally they are defined by:

- `StringVar(value="default value")`
- `IntVar(value=default_value)`
- `DoubleVar(value= default_value)`

For example to change the default value of dX, dY, dZ:

In `ScanPointFrame` class, in `_build(self)` function, in the for loop with `"enumerate(("X","Y","Z"))"`, there is:

`var = DoubleVar(value=0.1)`

If you want to set dX, dY and dZ to 1: replace this line by:

`var = DoubleVar(value=1)`

Add/delete oscilloscope settings on the interface ★★★★★

To do this, you need to understand how the different parts of the programs interact because you will have to edit 4 files 🤖.

What does `graphic_interface.py` do?

- Define everything displayed on the interface (buttons, text, etc)

- Display / hide some frame of interface
- Collects variables and passes them to exterior functions (in particular scan functions).

Warning: graphic_interface.py does not communicate directly with the oscilloscope.

So, to add oscilloscope setting on the interface, we will need to:

- Create buttons / labels / entry / etc
- Collect the new value and pass it to exterior function
- Modify the exterior function (to accept a new argument + take into account the new argument to set)

Next, I will explain how to add a setting like the “Sweep per acquisition”:

First: Go to OscSetting class (in graphic_interface.py). Then go to the end of build function and you will write these lines:

```
# Sweeps per acquisition
self.sweep_nb = IntVar(value=1)
ttk.Label(self, text="Sweep per acquisition :").grid(row=5, column=0,
sticky="e", padx=(0,4))
ttk.Entry(self, textvariable=self.sweep_nb, width=8).grid(row=5,
column=1, sticky="w")
```

Explanation:

- **L.1:** `#...` : comments about the setting you want to add
- **L.2:** `self.new_var = IntVar(value=1)` : create the variable you will give to your function (if you want to manipulate int: IntVar, string: StringVar, float: DoubleVar)
- **L.3:** `ttk.Label(self,...)` : create a label displayed in the interface.
- **L.3:** `.grid(row=5, column=0,...)` : define the position of your label, if you want to put it above all other settings: row= more than the others, column=0
- **L.4:** `ttk.Entry(self, ...)` : create a text entry where the user can enter a value.

PS: there are other widgets than Label or Entry, see the Tkinter documentation.

Now your new setting appears in the interface but is not connected to anything. So, you need to pass your new variable to the scan function. For that, search with Ctrl + F “sweep_nb” and add to the following functions your new variable (defined with IntVar()):

- `ScanPointFrame -> _on_click_launch(self) -> self.on_launch(...)`

- `ScanLengthFrame` -> `_on_click_launch(self) -> self.on_launch(...)`
- `ScanMatrixFrame` -> `_on_click_launch(self) -> self.on_launch(...)`
- `ScanPanel` -> `_launch_by_interf`
- `ScanPanel` -> `_launch_by_interf -> args`
- `ScanPanel` -> `_launch_matrix`
- `ScanPanel` -> `_launch_matrix -> args`

Ok, now you pass your variable to scan function but scans functions do not accept your new parameter yet. So, in `scan/scan_by_interf.py` and in `scan_for_matrix.py`:

- Add a new parameter in the defined function (`new_var=1` for exemple)
- Go to `osc_scan_settings` call and add your new var in parameter

Finally, open `scan/utils/osc_scan_settings.py`:

- Add your new var in parameter
- And write in code `osc.[setting_to_edit] = new_var`

And normally, now it should be ok.

Create a script to write a CSV file (for matrix mode) ★★☆☆☆

```
import csv
with open("file_name",mode="w",newline='') as f: #create or overwrite the file
    writer = csv.writer(f)
```

To create a csv file with a python script, you can copy this code. Then to write in your csv, you should use this command:

```
writer.writerow(list) # write list as a new row
```

So if `list=[1,2,3,4]`

After this line, your csv will contains the following row:

1,2,3,4

If you want you can use for loop to write csv. To help you, I will give you a little example:

```
import csv
with open("even_odd.csv", mode="w",newline='') as f: #create or edit your file
    writer = csv.writer(f)
    for i in range(200) :
        writer.writerow([i*2,i*2+1])
```

This script will create a csv file called `even_odd.csv` which contains in its first column even numbers up to 200 and odd numbers up to 201 in his second column.

The principle is the same for csv file for matrix mode, you just need to respect the format:

X,Y, Z (, R)

Create a script to view a scan result ★★☆☆☆

```
T = []
Y = []

with open('baguette_28/baguette_28_(0.0,0.0,0.0).csv', 'r') as f:
    lines = f.readlines()[1:]
    for line in lines:
        elt=line.split(",")
        T.append(float(elt[0]))
        Y.append(float(elt[1]))
```

To visualize a waveform, you can open your csv with python:

with open('file path', 'r') as f

Read all the data into a list:

Lines = f.readlines()[1:] # slicing to skip first row

And store all the values from first column in a specific list and do the same for second column:

➔ Watch for loop

And now you have two lists which allow you to plot the waveform with matplotlib:

```
plt.figure
plt.plot(T,Y)
plt.xlabel("Time (μs)")
plt.ylabel("Amplitude (V)")
plt.title(f"Waveform - {HDO4034A.MODEL_NAME} C1")
plt.grid(True)
plt.tight_layout()
plt.show()
```

PS: an example already exists in scan/utils/scan_viewer.py

Editing active axes with correct ID ★★☆☆☆

First, make a copy of hydrophone_handling folder.

Then, open hydrophone_handling/classes/ps90/OWIS_PS90.py: in open_connection fuction there are:

```
# Initialization of motorized axes
self.X = LinearStageL84N(self, axis_id=1) # Translation X → ...
self.Y = LinearStageL84N(self, axis_id=2) # Translation Y → ...
self.Z = LinearStageL84N(self, axis_id=3) # Translation Z → ...
self.R = RotaryStageDMT65(self, axis_id=4) # Rotation R → ...
```

Make sure that:

- translation axes use-> LinearStageL84N(self, second_argument)
- rotation axes use-> RotaryStageDMT65(self, second_argument)

Then, the second argument corresponds to the axis ID.

Next, if you want to use interface, open graphic_interface.py and rename the axis labels (in App class > _build_main_panel > for col, (axis, label, unit) in enumerate...).

Rename the axis labels and update the units if necessary.

```
for col, (axis, label, unit) in enumerate([
    (self.ps.X, "X", "mm"),
    (self.ps.Y, "Y", "mm"),
    (self.ps.Z, "Z", "mm"),
    (self.ps.R, "R", "°"),
]):
```

Now, it should work.

Launch a scan without interface ★★★★★

Use the file: scan/scan_classic.py

It was written for an older version of the project but normally it works.

You can try to adapt this file to create your scan.

Honestly, I don't have very helpful advises except to use the rest of the code as an example and look through the imported modules to understand how the scan works (and do not modify the interface-related code).

Other questions:

- ➔ Claude / Gemini / Chat GPT / Mistral / ...
- ➔ E-mail to timothee.chemla@free.fr